

Implement Client Script & UI Policy (Incident)

ServiceNow Implementation Documentation

Date	03 October 2026
Team members	Priyankha Sri J ,Abinaya R, Monika V
Project Name	Implement Client Script & UI Policy (Incident)
Platform	ServiceNow (Incident table)

1. Objective

Control how the Incident form behaves in the browser without server round trips. A **UI Policy** makes the On hold reason field visible and mandatory when an incident is On Hold. A **Client Script** reacts to Priority changes and makes Assignment group mandatory for critical (Priority 1) incidents.

2. Requirements Summary

#	Requirement	Mechanism
R1	When State is On Hold, On hold reason is visible and mandatory.	UI Policy
R2	When State is not On Hold, On hold reason is hidden and not mandatory.	UI Policy (Reverse if false)
R3	When Priority is 1 - Critical, Assignment group is mandatory and a message is shown.	Client Script (onChange)
R4	When Priority changes away from 1, mandatory flag and message are cleared.	Client Script (onChange)

3. Solution Architecture

The diagram below shows how a user's action on the Incident form is handled by the client-side UI Policy and Client Script, and how the data then flows to the server, the Incident table and downstream processes.

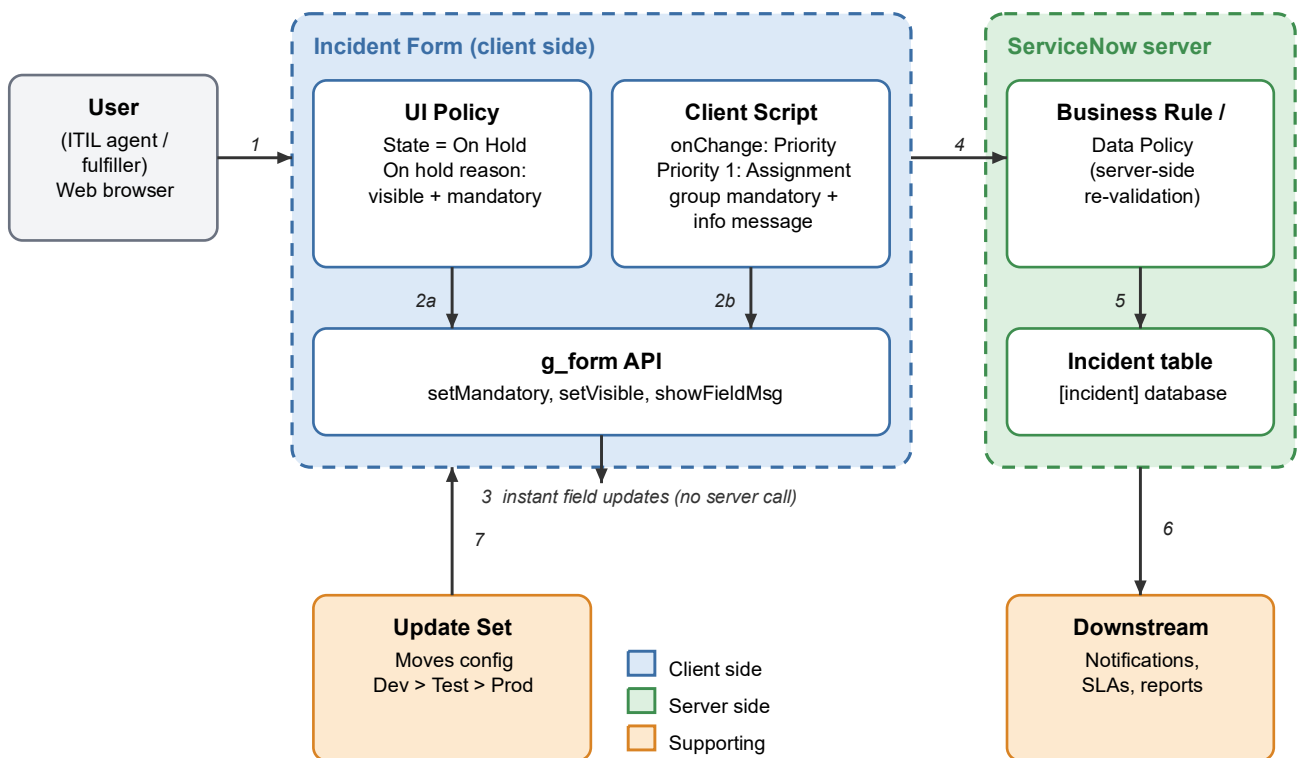


Figure 1: Architecture and data flow of the Incident Client Script and UI Policy implementation

Data flow

Step	Description
1	The user opens an Incident and changes State or Priority in the browser.
2a	The UI Policy evaluates State. If On Hold, On hold reason becomes visible and mandatory.
2b	The onChange Client Script evaluates Priority. If 1, Assignment group becomes mandatory and a message is shown.
3	Both use the g_form API to update the form fields instantly, with no server call.
4	On Save, the form is submitted. A Business Rule or Data Policy re-validates the same rules on the server.
5	Valid data is written to the Incident table.
6	Notifications, SLAs and reports run from the saved record.
7	The configuration is moved between instances (Dev, Test, Prod) with an Update Set.

4. UI Policy Implementation

Navigate to **System UI > UI Policies > New** and configure:

Field	Value
-------	-------

Table	Incident [incident]
Short description	Require On hold reason when State is On Hold
Active	True
Conditions	State is On Hold
On load	True
Reverse if false	True

UI Policy Actions

Save the policy, then add the following in the **UI Policy Actions** related list:

Field name	Mandatory	Visible	Read only
On hold reason	True	True	Leave alone
Resolution notes (optional)	Leave alone	False	Leave alone

5. Client Script Implementation

Navigate to **System Definition > Client Scripts > New** and configure:

Field	Value
Name	Incident - Priority change handler
Table	Incident [incident]
Type	onChange
Field name	Priority
Active	True

Script

```
function onChange(control, oldValue, newValue, isLoading, isTemplate)
{
    if (isLoading || newValue === '') { return; }

    // Priority 1 - Critical    if (newValue === '1') {
    g_form.setMandatory('assignment_group', true);
    g_form.showFieldMsg('priority', 'Critical incident:
    assign a group immediately.', 'info');    } else {
    g_form.setMandatory('assignment_group', false);
    g_form.hideFieldMsg('priority', true);    }
}
```

How it works

- **isLoading** is true while the form first loads; returning early avoids firing the logic before the user changes anything.
- **g_form.setMandatory()** toggles the red asterisk and blocks save when empty.
- **g_form.showFieldMsg()** / **hideFieldMsg()** display or clear an inline message under the Priority field.

6. Test Cases

ID	Steps	Expected Result
----	-------	-----------------

T1	Open a new Incident. Set State to On Hold.	On hold reason appears and is mandatory. Saving without it is blocked.
T2	Change State from On Hold to In Progress.	On hold reason is hidden and no longer mandatory.
T3	Set Priority to 1 - Critical.	Info message appears under Priority. Assignment group becomes mandatory.
T4	Change Priority from 1 to 3.	Message disappears. Assignment group is no longer mandatory.
T5	Open an existing Priority 1 incident.	No message on load (isLoading guard). Form opens normally.

7. UI Policy vs Client Script

Aspect	UI Policy	Client Script
Best for	Show, hide, mandatory, read-only	Custom logic, messages, calculations
Coding needed	Mostly none (point and click)	JavaScript required
Reverse logic	Built in (Reverse if false)	Must be coded manually
Runs on	Form load and field change	Form load, change, submit or cell edit

8. Best Practices and Notes

- Prefer a UI Policy for simple visibility and mandatory rules; use a Client Script only when scripting is needed.
- Client-side rules only affect the browser form. Back critical rules with a **Business Rule** or **Data Policy** so imports and APIs obey them too.
- Always include the **isLoading** check in onChange scripts.
- Test in a sub-production instance and capture the changes in an Update Set.
- Avoid duplicating the same rule in both a UI Policy and a Client Script; the two can conflict.

9. Conclusion

The UI Policy enforces the On hold reason rule declaratively, and the onChange Client Script adds Priority-driven behavior. Together they improve data quality on the Incident form with no impact on server performance.